Digital VLSI Design: 4-bit Array Multiplier from RTL to GDSII using Open-Source Tools

Team Members:
Lakshya Jindal (2023112017) & Sai Ritvik Uppuganti (2025122012)

October 25, 2026

Abstract

This report documents the RTL-to-GDSII implementation of a 4-bit × 4-bit unsigned array multiplier (8-bit product) on the SkyWater 130 nm process (`sky130_fd_sc_hd`): RTL design and exhaustive verification with Icarus Verilog, logic synthesis with Yosys, static timing analysis with OpenSTA, and place and route with LibreLane. All numbers come from the reports and metrics files in the repository. Setup timing closes at a clock period of 7.0 ns in every corner, with 0 DRC and 0 LVS errors.

Contents

1 Introduction **2**

2 Part A: RTL Design and Verification **2**

3 Part B: Logic Synthesis with Yosys **3**

4 Part C: Static Timing Analysis with OpenSTA **4**

5 Part D: Physical Design (RTL to GDSII) **5**

6 Answers to Report Questions **6**

7 Conclusion **7**

8 Contributions **7**

1 Introduction

The design is a 4-bit unsigned array multiplier with registered inputs and a registered output. It has 16 AND gates for the partial products, 12 full adders arranged as three ripple-carry rows, and 16 flip-flops, so the latency is 2 clock cycles and the throughput is one result per clock cycle. Tools (all inside IIC-OSIC-TOOLS): Icarus Verilog 14.0 (devel), Yosys 0.69 (git 9f75ca1f9), OpenSTA 3.1.0, and LibreLane v3.1.0.dev3 with OpenROAD, Magic, KLayout and Netgen. Process and library: SKY130A, `sky130_fd_sc_hd`, typical corner `tt_025C_1v80` (typical process, 25 °C, 1.80 V) for Parts B and C. The work was done as a team project of two members, and Section 8 lists who did which part.

2 Part A: RTL Design and Verification

Commands (from the repository root):

```
iverilog -g2012 -o sim tb/tb_mult.v rtl/full_adder.v rtl/mult_array.v
vvp sim
gtkwave mult.vcd
```

Verification summary line: `PASS: 256 vectors checked, 0 errors.`

Error injection (Task 6): changing `sum` in `full_adder.v` to `a ^ b` (ignoring `cin`) makes the testbench print `FAIL: 256 vectors checked, 95 errors`. The change was then undone and the testbench returned to PASS.

Hand-worked example (Task 5). The roll number 2023112017 ends in 7, so $A = 7 + 5 = 12$ (1100_2) and $B = 7$ (0111_2). The partial products are $A \cdot b_0 = 1100$, $A \cdot b_1 = 1100$ shifted by 1, $A \cdot b_2 = 1100$ shifted by 2 and $A \cdot b_3 = 0000$ (because $b_3 = 0$), and they add up to $P = 12 \times 7 = 84$ (01010100_2, `0x54`).

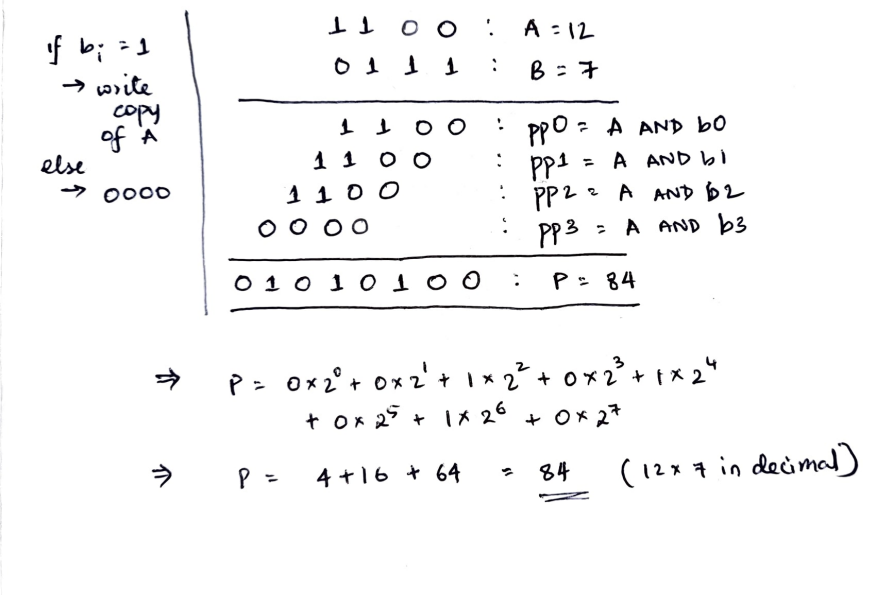


Figure 1: Hand-worked multiplication 12×7 in the layout of Section 2 of the guide: a partial product $A \cdot b_i$ is a copy of A if $b_i = 1$ and 0000 otherwise, each shifted left by i places. The sum is $P = 01010100_2 = 4 + 16 + 64 = 84$.

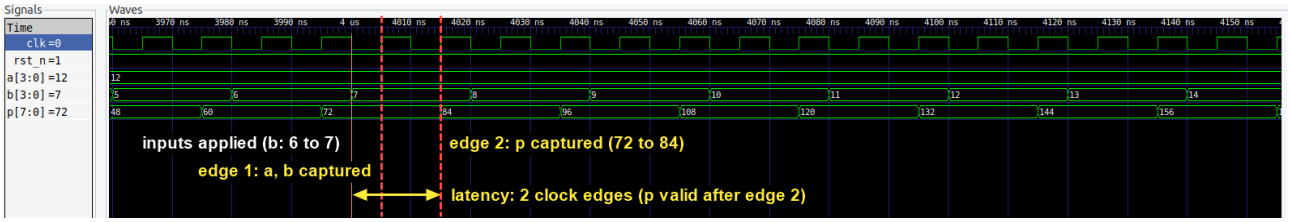


Figure 2: GTKWave screenshot of the simulation for $A = 12$, $B = 7$ (10 ns clock period; markers added). `b` changes from 6 to 7 at the marked time, the input registers capture `a` and `b` at edge 1, and `p` changes from 72 (12×6) to 84 (12×7) at edge 2: a latency of 2 clock cycles.

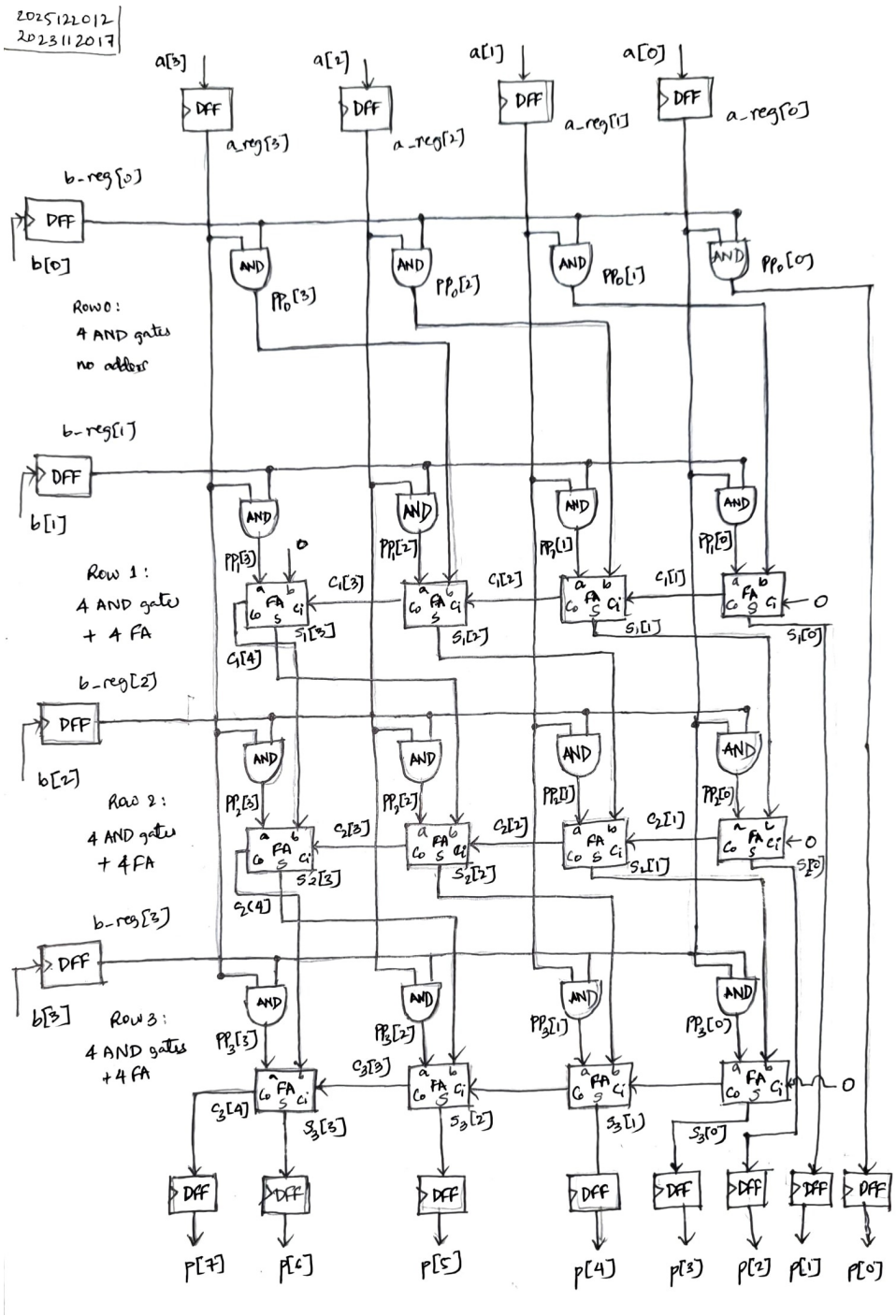


Figure 3: Hand-drawn block diagram of the multiplier (scanned): 8 input flip-flops (`a_reg`, `b_reg`), the 16 AND gates that give the partial products $pp_i[j] = \texttt{a_reg}[j] \wedge \texttt{b_reg}[i]$, 12 full adders (FA) in three rows of four, and 8 output flip-flops. Row 0 has no adders. $s_i[j]$ and $c_i[j]$ are the sum and carry of adder j in row i, and $c_i[4]$ is the carry out of the leftmost adder of the row. The adder inputs tied to constant 0 are the carry-in of the rightmost adder in each row and the b input of the leftmost adder in row 1. The product bits are $p[0] = pp_0[0]$, $p[1] = s_1[0]$, $p[2] = s_2[0]$, $p[3] = s_3[0]$ and $p[7{:}4] = \{c_3[4], s_3[3], s_3[2], s_3[1]\}$. **Not drawn:** the clock `clk` (rising edge) and the asynchronous active-low reset `rst_n`, which go to all 16 flip-flops and reset them to 0; only the clock triangle is marked on each flip-flop and the two wires are left out to keep the figure readable.

3 Part B: Logic Synthesis with Yosys

Yosys maps the RTL to SKY130 cells using `synth -top mult_array -flatten`, `dfflibmap`, `abc` and `opt_clean`. Command: `yosys -c synth/synth.tcl | tee synth/yosys.log`. The netlist was simulated with the same testbench using `iverilog -g2012 -DFUNCTIONAL -DUNIT_DELAY=##1` and the SKY130 cell models. The gate-level simulation confirmed functional equivalence: all 256 vectors pass, so synthesis did not change the behaviour.

Table 1: Part B synthesis results (`synth/mult_array_stat.rpt`).

Metric	Value
Total number of cells	68
Number of flip-flops (`dfrtp_1`)	16
Total cell area (μm^2)	758.23
Flip-flop area (μm^2)	400.38
Flip-flop area (% of total)	52.81
Three most-used cell types	`dfrtp_1` (16), `nand2_1` (13), `a21oi_1` (4)
Netlist simulation result	`PASS: 256 vectors checked, 0 errors`

4 Part C: Static Timing Analysis with OpenSTA

OpenSTA is run at the typical corner (`tt_025C_1v80`) with the commands `sta sta/sta.tcl` ($T = 10$ ns, report kept as the deliverable), `T=5.0 sta sta/sta.tcl` and `T=2.75 sta sta/sta.tcl`. The worst slack was 7.253 ns at $T = 10$ ns, 2.253 ns at $T = 5$ ns and 0.003 ns at $T = 2.75$ ns. It changes by exactly the change in T, so $T_{\min} = T - \text{WNS} = 10 - 7.253 = 2.747 \approx 2.75$ ns.

Table 2: Part C timing results (typical corner, ideal clock, 1.0 ns input and output delays).

Metric	Value
Worst slack at $T = 10$ ns	7.25 ns
$T_{\min}$	2.75 ns
$F_{\max} = 1000/T_{\min}$	363.64 MHz
Slack at $T = T_{\min}$ (2.75 ns)	0.00 ns (0.003 ns exact)
Critical path start point	`_104_` (flip-flop `a_reg[0]`)
Critical path end point	`_102_` (flip-flop `p[6]`)
Number of cells on the critical path	8 combinational cells
Latency ($2 \times T_{\min}$)	5.50 ns
Throughput	363.64 million results/s

Table 3: Critical path from `sta/mult_array_timing.rpt`, in order (cell delays in ns).

#	Instance	Cell	Delay	Arrival	Role
0	`_104_`	`dfrtp_1`	0.378	0.378	launch flip-flop `a_reg[0]` (clock-to-Q)
1	`_045_`	`and2_0`	0.185	0.563	partial-product AND
2	`_051_`	`o21a_1`	0.212	0.775	merged adder logic
3	`_054_`	`or4_1`	0.496	1.272	merged adder logic
4	`_075_`	`a21bo_1`	0.209	1.480	merged adder logic
5	`_081_`	`maj3_1`	0.369	1.850	full-adder carry (majority)
6	`_085_`	`xnor3_1`	0.209	2.059	full-adder sum (XNOR3)
7	`_086_`	`maj3_1`	0.379	2.438	full-adder carry (majority)
8	`_088_`	`xnor3_1`	0.175	2.613	full-adder sum, row 3 column 3 (into `p[6]`)

Library setup time of the capture flip-flop `_102_`: 0.134 ns.

Full-adder content of the path (Task 3). Of the 8 combinational cells, 4 come from full adders: the two `maj3_1` cells are carry (majority) functions and the two `xnor3_1` cells are sum (3-input XNOR) functions, and the last one drives the sum bit of row 3, column 3 into `p[6]`. One cell (`and2_0`) is a partial-product AND, and the remaining 3 (`o21a_1`, `or4_1`, `a21bo_1`) are logic that ABC merged from the early adder rows. The guide estimates about $2N = 8$ full-adder delays. The mapped path also has 8 cells, but only 4 of them are full-adder cells, so the cell count and the adder-stage count agree by coincidence and not one-to-one.

5 Part D: Physical Design (RTL to GDSII)

LibreLane ran the full flow (synthesis, floorplan, placement, clock tree synthesis, routing, signoff, GDSII) on a fixed $150 \times 150\,\mu$m die (`FP_SIZING = "absolute"`, `DIE_AREA = [0, 0, 150, 150]`). The first clock period was $1.2\,T_{\min} = 3.3$ ns rounded up to 3.5 ns. Following the guide, the period was increased by 0.5 ns after every run whose worst setup slack (`timing__setup__ws`, the minimum over all nine corner entries) was negative, until it was ≥ 0 with 0 DRC and 0 LVS errors. In the runs whose metrics were inspected (6, 7 and 8), the worst corner is the slow one (`max_ss_100C_1v60`: slow process, 100 °C, 1.60 V).

Table 4: Clock period tuning (setup slack in ns). Runs 1 to 5 are as logged in the repository README, runs 6 to 8 are from the metrics files.

Run	`CLOCK_PERIOD` (ns)	Worst setup slack (ns)	Typical-corner slack (ns)
1	3.5	−1.8011	0.6167
2	4.0	−1.3195	1.1132
3	4.5	−0.8192	1.6134
4	5.0	−0.3192	2.1134
5	5.5	−0.0712	2.4658
6	6.0	−0.0443	2.7877
7	6.5	−0.0347	3.3395
8	7.0	+0.4653	3.8395

Table 5: Part D results (final run, `pnr/metrics.json`, `CLOCK_PERIOD` = 7.0 ns).

Metric	Value
`CLOCK_PERIOD` used	7.0 ns
Number of runs needed	8 (3.5 to 7.0 ns in steps of 0.5 ns)
Core area	17759.5 μm^2 (die $150 \times 150\,\mu$m = 22500 μm^2)
Cells after synthesis (Part B)	68
Cells after place and route	1883 in total: 83 logic, flip-flop and buffer cells, 245 tap, 1299 decap and 256 fill cells
Kinds of cells added	tap, decap and fill cells, 3 clock buffers (`clkbuf_16`), 17 port buffers (`buf_2`, one per input and output port) and 2 fanout buffers (`clkdlybuf4s50_1`)
Wirelength	2333 μm
Setup slack	+0.465 ns worst corner (`max_ss_100C_1v60`); +3.839 ns typical corner
Post-route $F_{\max} = 1000/(\text{period} - \text{slack})$	153.03 MHz (worst corner); 316.40 MHz (typical corner)
$F_{\max}$ from Part C	363.64 MHz
DRC errors	0 (Magic 0, KLayout 0, routing 0)
LVS errors	0 (Netgen); XOR difference 0; antenna violations 0

Hold slack is +0.113 ns with no hold violations, and there are no setup violations in any corner.

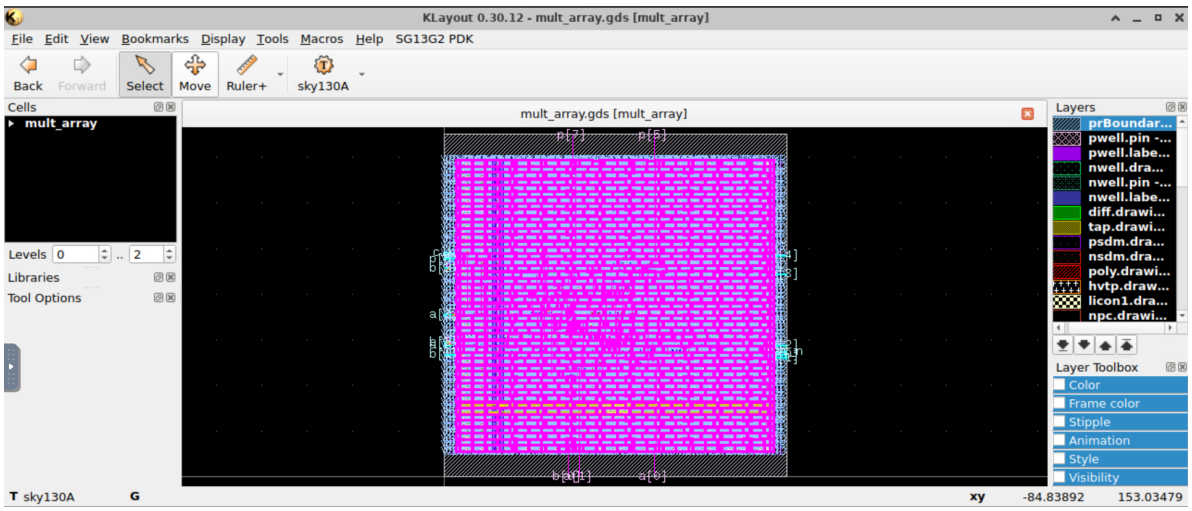


Figure 4: Whole die ($150 \times 150\,\mu$m): KLayout screenshot of `pnr/mult_array.gds`, the 7.0 ns layout submitted, with the sky130A layer set. The standard-cell core fills the die and the I/O pins sit on its edges.

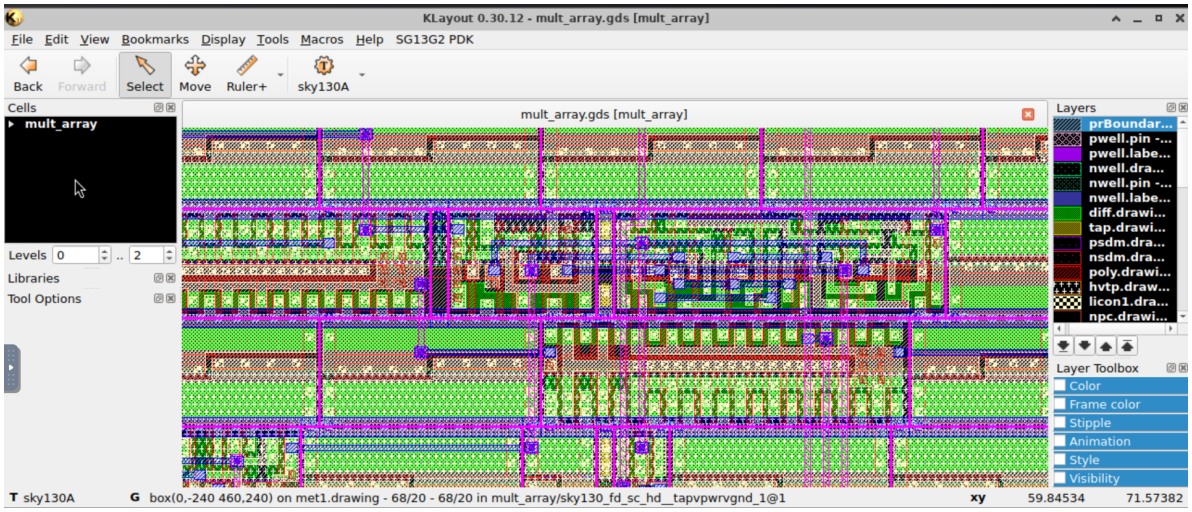


Figure 5: Zoomed view (about $21 \times 9\,\mu$m, three standard-cell rows): KLayout screenshot of the cell layers, the power rails along the row edges and the signal wires between the cells.

6 Answers to Report Questions

1. Logic cells. Synthesis produced 52 logic cells (68 total minus 16 flip-flops), against $16 + 12 = 28$ in the structural picture. First, ABC only maps to single-output cells. The SKY130 library does contain multi-output adder cells such as `sky130_fd_sc_hd__fa_1`, but ABC skips them (the Yosys log reports 9 multi-output cells ignored) and the netlist contains none, so every full adder needs at least two cells, one for the sum and one for the carry. Second, after `-flatten` ABC restructures logic across the former adder boundaries and covers it with a mix of NAND2 (13 cells), AND-OR-INVERT and OR-AND cells and XOR/XNOR cells (8 in total) instead of one cell per original gate. Third, some adder inputs are constant 0 (the carry-in of each row and one input of row 1), so those adders reduce to simpler logic.

2. Critical path. The critical path runs from the input register `a_reg[0]` (cell `_104_`) to the output register bit `p[6]` (cell `_102_`) through 8 combinational cells, with a data arrival time of 2.613 ns (Table 3). In order, the cells are `and2_0`, `o21a_1`, `or4_1`, `a21bo_1`, `maj3_1`, `xnor3_1`, `maj3_1` and `xnor3_1`. Four of them (the `maj3_1` and `xnor3_1` cells) are full-adder carry and sum cells, one is the partial-product AND, and three are logic that ABC merged from the early adder rows instead of keeping separate full adders. The guide's estimate is about $2N = 8$ full-adder delays. The path has 8 mapped cells, but only 4 of them are full-adder cells, so the two numbers agree by coincidence and do not match one-to-one: ABC merged several adder stages into larger cells such as `or4_1`. Dividing the 2.235 ns of combinational delay by 8 gives about 0.28 ns per full-adder-equivalent delay.

3. Flip-flop share of the area. The 16 flip-flops are 400.38 μm^2 of 758.23 μm^2, which is 52.81 %, although

they are only 16 of the 68 cells. One `dfrtp_1` is 25.024 μm^2 (20 sites of 1.2512 μm^2), whereas a `nand2_1` is 3.754 μm^2 (3 sites), so a flip-flop is about 6.7 times larger. The design needs only 12 adders and 16 ANDs of small combinational logic, but the architecture requires 16 flip-flops, each containing master and slave latches, internal clock inverters and the asynchronous reset.

4. Post-route $F_{\max}$ and added cells. The post-route $F_{\max}$ is 153.03 MHz at the worst corner (316.40 MHz at the typical corner), lower than the 363.64 MHz of Part C. Part C is a pre-layout estimate at the typical corner with an ideal clock and no wires. The post-route analysis uses the slowest corner (100 °C, 1.60 V), where the worst post-route path (flip-flop `_90_` to `_88_`, 8 cells) takes 5.97 ns from the launch flip-flop clock pin against 2.61 ns in Part C, with a clock-to-Q of 0.965 ns against 0.378 ns. It also includes wire resistance and capacitance, the clock tree, 0.25 ns of clock uncertainty and a 0.29 ns setup time. The typical-corner post-route figure (316.40 MHz) is much closer to Part C, so most of the gap comes from the slow corner. The flow added: tap cells (245), which tie the wells and substrate to the supplies and prevent latch-up; decap cells (1299), which add local charge storage against supply droop; fill cells (256), which close the gaps in the cell rows so that wells and power rails are continuous; clock buffers (3 `clkbuf_16`), which distribute the clock to the 16 flip-flops (the tree is not zero-skew: the worst clock skew is 0.26 ns for setup and 0.29 ns for hold); port buffers (17 `buf_2`, named `input1` to `input9` and `output10` to `output17`), one on each of the 9 data and reset inputs and 8 outputs, which isolate the ports from the internal nets; and 2 fanout buffers (`clkdlybuf4s50_1`, named `fanout18` and `fanout19`), which the resizer inserts to limit fanout. None of the added buffers is a hold fix: the hold slack is +0.113 ns with no hold violations.

5. DRC and LVS. Design Rule Checking verifies that every shape of the layout (widths, spacings, enclosures, overlaps) obeys the foundry's manufacturing rules for SKY130, so the layout can be fabricated with acceptable yield. Layout Versus Schematic extracts the transistor-level netlist from the layout and compares it with the synthesized gate-level netlist, to confirm that the drawn circuit is the intended one with no shorts, opens or missing devices. A layout can be rule-clean yet implement the wrong circuit (DRC passes, LVS fails), or be the right circuit yet unmanufacturable (LVS passes, DRC fails), so both must show 0 errors before tape-out.

6. An 8-bit multiplier. With the formulas of Section 2 of the guide, an N-bit array multiplier has N^2 AND gates and $N(N-1)$ full adders, and a longest path of about $2N$ full-adder delays. For $N = 8$: $8 \times 7 = 56$ full adders (against 12, 4.67 times as many), $8^2 = 64$ AND gates (against 16) and about $2 \times 8 = 16$ full-adder delays (against 8, twice as long). So the area grows roughly quadratically and the delay linearly. As a rough estimate with the Part C numbers, the 8 combinational delays of 2.235 ns (2.613 minus the 0.378 ns clk-to-Q) give about 0.28 ns per adder delay, so 16 of them take about 4.47 ns and $T_{\min} \approx 0.378 + 4.47 + 0.134 \approx 5.0$ ns for 8 bits against 2.75 ns for 4 bits.

7 Conclusion

The 4-bit array multiplier was designed, verified (256 of 256 vectors, in RTL and on the synthesized netlist), synthesized (68 cells, 758.23 μm^2), timed ($T_{\min}$ = 2.75 ns, $F_{\max}$ = 363.64 MHz before layout) and taken to a GDSII layout. At `CLOCK_PERIOD` = 7.0 ns the layout has a worst setup slack of +0.465 ns over all corners, hold slack +0.113 ns, and 0 DRC and 0 LVS errors; its post-route $F_{\max}$ is 153.03 MHz at the slowest corner.

GitHub repository: https://github.com/lakshya0310/RTL_To_GDS_Assignment

8 Contributions

- **Lakshya Jindal (2023112017):** Part A (RTL design and verification): the Verilog for the full adder and the array multiplier, the self-checking testbench, the hand-worked example for his roll number and the GTKWave check of the waveform. Part D (physical design): the LibreLane configuration and the first six clock-period runs (3.5 ns to 6.0 ns), recorded in the repository README.
- **Sai Ritvik Uppuganti (2025122012):** Part A: the hand-drawn block diagram of the multiplier (16 AND gates, 12 full adders and 16 flip-flops). Part B (logic synthesis): the Yosys script, the analysis of cell area and flip-flop share, and the gate-level simulation of the netlist. Part C (static timing analysis): the OpenSTA script, the critical path, $T_{\min}$ and $F_{\max}$ at the typical corner. Part D follow-up: the runs at 6.5 ns and 7.0 ns that brought the worst setup slack to ≥ 0 in all corners, and the final `metrics.json` and GDS in the repository.